

Challenge 4 — 8 minute talk

Four parts, matching the speaker's talking points. Timings are a guide, and they add up to about seven and a half minutes so there is room to breathe.

1 · What I chose to grow

≈50 seconds

My challenge was: **learn to build connected data tracking across watchOS and iOS.**

My growth focus statement said I would learn how that works – background execution, data ownership, transport reliability, source of truth – by producing one complete workout captured on the watch and reviewed on the phone, **together with a log of what I predicted wrongly**, so that I could build a data crossing that holds through screen-off and a force quit, and explain why it holds.

That last part was deliberate. **A log of what I predicted wrongly.** I will come back to it.

I train HYROX. Eight runs, eight stations, and the rest in between counts. I wanted an app that records the race on the watch and explains it honestly on the phone.

2 · My starting point, and where I am now

≈1 minute 30

My starting point was zero. HealthKit, workout sessions, background execution, moving data between two devices – I had used none of them.

So I did not plan to build first. **I planned to predict first.**

Before every experiment I wrote down what I expected. After it ran, I wrote what happened, and the gap between them. **A prediction can never be edited after the test.** A wrong prediction is the evidence, not a mistake to hide.

Ten days later: six cycles, twenty-four experiments, all on a real Apple Watch and a real iPhone. A watch app that records a race and survives a force quit. A phone app that reviews it. And a two-thousand-line workbook with every wrong prediction still in it.

Here is what that method bought me. Cycle 4 was planned as the **largest build in the project** – a connection to move data from the watch to the phone. **I built none of it.** Cycle 3 had already measured that Apple moves it for free: under fourteen seconds, phone locked, in another room. So

Cycle 4 became a decision instead of a build, and I wrote a document explaining why building it would be waste.

The largest planned task in my project produced zero code. That is the result I am most confident about, because I only knew it was safe to delete after measuring it.

3 · What and how I learned

≈3 minutes 30 – the main part

One moment, because it changed everything after it.

The 2.03 second rest

In Cycle 5 I stopped tapping through tests at my desk and ran a **real session**. Real running, real burpees. I was out of breath, pressing a button on my wrist between stations.

At the second rest, I **pressed it twice by accident**.

My app recorded a rest of **2.03 seconds**. That is impossible.

And every check I had written said the data was **fine**. Timestamps in order. Nothing negative. Everything inside the workout. My integrity check reported INTACT.

What I learned from it

I had been treating all my numbers as equally solid. They are not. There are four kinds:

KIND	WHAT IT MEANS
MEASURED	The watch recorded it alone – heart rate, energy, total time
MARKED	A human pressed a button – every station boundary
DERIVED	Worked out from those – every station duration
UNSUPPORTED	The data cannot answer it at all

The obvious split would be recorded versus calculated. **That split is wrong**. The real one sits inside "recorded." My total time and my station start time are both timestamps and look identical on screen. One came from a sensor. The other came from **a tired person pressing a button**.

Then I found a fifth kind no plan of mine contained. My app printed "**SkiErg**" when I had done burpees – because it **assumed** the station and never asked. I called that **ASSUMED**.

The pattern underneath

Once I saw it, I saw it everywhere. **A green signal hiding a failure – seventeen times, and I have indexed every one.**

- The app looked healthy while losing **79%** of the race, with no crash and no error.
- Permission reported success while reading was blocked, because HealthKit returns an empty list instead of an error.
- My integrity check passed that impossible two-second rest.
- My comparison tool printed **AGREE** when it had compared nothing.

Several were inside **the tools I built to catch problems**. Two were **my own mistakes** – I once concluded HealthKit had deleted all my workouts, and they came back the same day, unchanged. That retraction is still in my workbook.

What I learned, in one sentence: a passing test only proves what it actually checked.

So my checks now say what they looked at. Instead of `AGREE`, my tool prints:

```
AGREE – 6 durations compared; the final one skipped, because there is nothing
after it to compare against
```

One thing that gave me confidence

Late on, I checked **Apple's own Workout app**. I pressed its lap button twice, **0.333 seconds apart**. It recorded both, no complaint, no flag.

Apple has not solved this either. So this is not me being a beginner. Solving it properly is a real contribution.

4 · Learning Evidence

≈ 1 minute 30

The app – a watch app that records a HYROX race and survives a force quit, and a phone app that labels where every number came from.

The workbook – two thousand lines. Every prediction before the test, every result after, every gap between. Including two conclusions I had to retract.

The decision records – three documents, including the one that says do not build the transport, with the conditions that would reverse it.

The commits – every finding is a commit with the reasoning in the message.

On the device, live: three clocks side by side during a workout. One counts by timer and falls behind when the watch suspends the app. Two count from timestamps and stay correct. **That one screen is my entire first cycle, happening in front of you.**

And one line of code. There is no HYROX workout type in HealthKit. I added a single metadata key, and **Apple's own Health app now shows my workout as HYROX** – while I keep my own live screens.

Where I go next

I am designing the screens on paper in Sketch **before** writing more interface code. My two worst problems are design problems that paper would have caught.

I built an undo, and **when I made the mistake mid-race, I did not use it** – because it was on a different screen.

And my app still tells the athlete what station they did, instead of asking.

Thank you.

Appendix — hard questions, and honest answers

Not spoken. This is preparation for questions, including the ones a hostile reader would ask. Where the honest answer is a limitation, it is written as a limitation.

"You barely built anything. Where is the app?"

Two apps, on real hardware. A watch app that records a HYROX race, survives a force quit, and writes structure as `HKWorkoutEvent`s. A phone app that reads them back and labels the provenance of every number. Both signed, installed and run on an Apple Watch Ultra 3 and an iPhone 16 Pro Max – never on a simulator.

What I did *not* build is a watch-to-phone transport, and that was a decision with a written record, not an omission.

"Deciding not to build the biggest task sounds like avoiding work."

It would be, if the decision came first. It came after three measurements: E2.3 showed offsets remove reconciliation entirely, E3.2 showed custom metadata crosses on its own, E4.0 showed a full-length

race payload – 25 segments, 601 characters – crosses without truncation.

The decision record at `design/L4-transport-decision.md` lists the reversal conditions. If any of them turn out to hold, the decision flips. That is what separates a decision from an excuse.

"Two of your conclusions were wrong. Why should I trust the others?"

Because you can see both. B1 and B2 in the evidence index are my errors, retracted **in place**, with the original wrong text kept above the retraction.

If I had removed them you would have no way to audit my reasoning. The fact that the record contains its own corrections is the reason to trust the rest of it – not a reason to doubt it.

"How do I know the predictions were really written before the tests?"

Git history. Every prediction is a commit that precedes the commit carrying its result. The workbook rule is stated at the top of the file: a prediction is never edited after the test runs.

That is checkable, and it is the only reason the wrong predictions are worth anything.

"Your app loses 1.2% of the race. That is not accurate enough."

It does not. That figure measures how often the app was **allowed to run**, not the accuracy of anything recorded. Every time value in the product path is a subtraction between two timestamps –

`Date().timeIntervalSince(...)` – and `ProtocolMachine` contains no tick counting at all.

The 1.2% comes from a counter built deliberately to measure suspension. The recorded race has never carried any drift, and since 2026-09-04 the on-screen clock is rendered by the system, so the display carries none either.

"Why not use WorkoutKit? Apple built it for this."

I evaluated it. `CustomWorkout` with a `displayName` would show the name HYROX, and would replace my live screens with Apple's interval interface – the state machine, the advance control and the correction flow all go away.

One metadata key achieved the same naming while keeping all of it. Apple's Health app now displays the workout as HYROX. The trade was one line of metadata against the entire live experience.

"Sample size. One athlete, a handful of sessions."

Correct, and it limits some claims and not others.

Not limited: the platform behaviours. 79% loss without a session, recovery not being idempotent, atomicity across three injected crash points, metadata crossing at full race length. These are properties of watchOS and HealthKit, reproducible by anyone.

Limited: anything about how athletes behave. That a tired person mistaps is evidenced by one person mistapping once. It is a real observation and it is not a study.

"One watch, one phone, one OS version."

True, and stated as a risk in the L4 decision record. Nothing here has been tested across watchOS versions, on older hardware, or on a watch not paired to its own phone. A 90-minute session has also never been run end to end; the longest real session was 5 minutes 11 seconds, and the full race was seeded rather than performed.

"The 2.03-second rest — why not just debounce the button?"

Debouncing hides it. If two presses 0.333 seconds apart are silently collapsed, the record shows one clean mark and nothing indicates a human was uncertain there.

That would create a value that reads as MARKED but was actually manufactured — the exact dishonesty the taxonomy exists to prevent. The design now surfaces it in review, states the evidence, and records that a correction was made. Apple's own Workout app does debounce nothing and flags nothing, so neither approach is the industry answer yet.

"Isn't the four-category taxonomy just labelling?"

It changed the code. `WorkoutReview` withholds `FROM MARKED + MEASURED` unless the final segment genuinely ends with the workout. `segmentSourceAgreement` reports how many durations it compared and names the one it skipped. The empty state on iOS refuses to claim there are no workouts, because a refused read and an empty store are indistinguishable.

Each of those is a sentence the app is no longer allowed to say.

"What did you actually learn, as opposed to what the tools did?"

The transferable thing is a habit, and it is testable on me: I now write down what I expect before I run anything, and I treat a passing result as a claim about the test rather than about the world.

The evidence that it took is that five of the seventeen indexed failures were found **inside instruments I had built to catch that exact failure** — and I kept finding them, including two on the last day, because I was looking.

"Why is the count seventeen when your earlier drafts said eleven?"

Because the earlier figures were incremented, never enumerated. The workbook names a "third", a "tenth" and a "twelfth" instance and never names an eleventh.

Building `EVIDENCE-INDEX.md` produced an auditable figure for the first time. The claim was right in shape and wrong in size, and it was wrong for exactly the reason the index is about: it had never been checked.